

Matrices associées à un graphe4

Matrice d'incidence sommets-Arcs	4
Exemple	4
Déclaration de la structure	5
Remarque	5
Matrice d'incidence sommets-Arêtes	6
Exemple :	6
Déclaration de la structure	6
Matrices d'adjacence A	7
Exemple	7
Remarque	8
Déclaration de la structure	8
Remarques	9
Propriétés	10
Théorème (voir TD).....	10
Représentations des graphes	11
Représentation à partir de la matrice d'adjacence.....	12
Remarque	12
Avantages.....	12
File des successeurs d'un graphe.....	14
Rappel : Fonction successeur	14
Exemple	15
Déclaration de la structure	16
Remarques	16
Représentation par liste d'adjacences.....	17

Exemple	17
Déclaration de la structure : représentation chaînée	18
Occupation mémoire.....	18
Traitement d'un successeur d'un sommet s :	19
Représentation Contigüe de la structure.....	19
Listes adjacences Successeurs & Prédécesseurs	20
Exemple	20
Déclaration de la structure : représentation chaînée	21
Remarques	23
Représentation par pointeurs= Liste principale.....	24
Exemple	25
Déclaration de la structure	26
Représentation de la matrice d'adjacence par une matrice creuse...27	
Définition	27
Principe	27
Exemple	27
Structure en langage C.....	28
Représentation statique.....	31
Rappel: Définition d'un Multigraphe	33
a)Représentation colonne par colonne	33
Exemple	34
Remarques	35
Déclaration de la structure	35
b) Description ligne par ligne de la matrice d'incidence.	36
Exemples.....	37

Remarque	39
Déclaration de la structure	39
Liste d'incidences	40
Exemples.....	40
File d'incidences.....	42
ETC	42
Choix entre les différents types de représentations.....	43

Matrices associées à un graphe

Matrice d'incidence sommets-Arcs

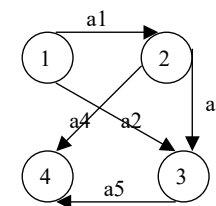
La matrice d'incidence sommets-arcs d'un graphe orienté $G=(S, A)$ est une matrice $M=(m_{sa})$, $s=1..n=|S|$, $a=1..p=|A|$ à coefficients entiers $\{0, +1, -1\}$ telle que chaque colonne **a** correspond à un arc de G , et chaque ligne **s** à un sommet de G .

Si $a=(s, t) \in A$, la colonne de a à tous ses termes nuls, sauf :

$$m_{sa}=+1$$

$$m_{ta}=-1$$

Exemple



	a1	a2	a3	a4	a5
1	1	1	0	0	0
2	-1	0	1	1	0
3	0	-1	-1	0	1
4	0	0	0	-1	-1

Matrice d'incidence Sommets-Arcs

Déclaration de la structure

```
typedef struct {
    int nbSom ;
    int nbArcs ;
    int **mat ;
    // TCout **cout ;
} MatIncidente ;
```

Remarque

*Lorsque le graphe est valué, c'est à dire qu'à chaque arc ou arête $a \in A$ on associe un poids ou une valeur $v(a)$, la valeur peut être stockée dans un tableau **cout**, en correspondance biunivoque avec mat*

Matrice d'incidence sommets-Arêtes

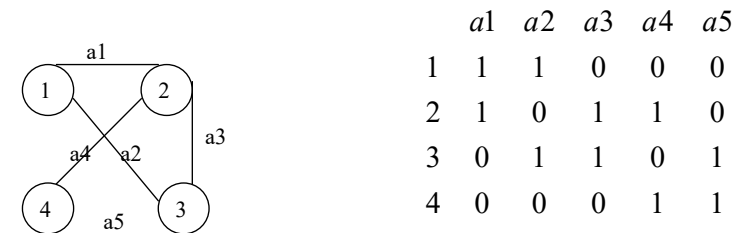
Soit G un graphe Non orienté $G=(S, A)$.

La matrice d'incidence sommets-arêtes de G est une matrice à coefficients 0 ou 1, où chaque ligne correspond à un sommet s de G et chaque colonne à une arête $a=\{s, t\} \in A$.

Si $a=\{s, t\} \in A$, alors la colonne a à tous ses éléments nuls sauf :

$m_{sa} = m_{ta} = 1$

Exemple :



Matrice d'incidence Sommets-Arêtes

Déclaration de la structure

```
typedef struct {
    int nbSom ;
    int nbAretes ;
    int **mat ;
    //TCout **cout ;
} MatIncidente ;
```

*Lorsque le graphe est valué, c'est à dire qu'à chaque arc ou arête $a \in A$ on associe un poids ou une valeur $v(a)$, la valeur peut être stockée dans un tableau **cout**, en correspondance biunivoque avec mat*

Matrices d'adjacence A

Rappel : 1-graphe

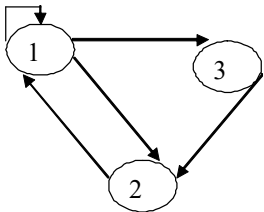
- Il n'existe pas plus d'un arc de la forme $(s, t) \forall s, t \in S$
- Comportant éventuellement une boucle par sommets.

Soit $G=(S, A)$ un 1-graphe

La matrice d'adjacence associée à G (matrice d'incidence sommets-sommets) est une matrice carrée d'ordre $n=|S|$ à coefficients 0 ou 1 :

$$A=(m_{st}) \text{ pour } s=1..n \text{ et } t=1..n \text{ on a : } m_{s,t}=\begin{cases} 1 & \text{ssi } (s,t) \in A \\ 0 & \text{sinon} \end{cases}$$

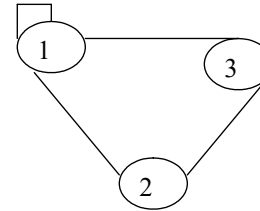
Exemple



	1	2	3
1	1	1	1
2	1	0	0
3	0	1	0

Remarque

Dans le cas d'un graphe non orienté, on peut aussi définir la matrice d'adjacence d'un graphe simple. Dans ce cas la matrice est symétrique. En effet, à chaque arête $\{s, t\}$, correspondent deux arcs (s, t) et (t, s)



	1	2	3
1	1	1	1
2	1	0	1
3	1	1	0

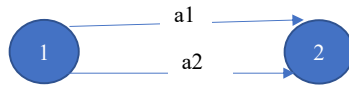
Déclaration de la structure

```
typedef struct {
    int nbSom ;
    int **mat ;
    //TCout **cout ;
} MatAdjacence ;
```

*Lorsque le graphe est valué, c'est à dire qu'à chaque arc ou arête $a \in A$ on associe un poids ou une valeur $v(a)$, la valeur peut être stockée dans un tableau **cout**, en correspondance biunivoque avec mat*

Remarques

- Représentation est efficace au niveau de l'espace mémoire, utilisée lorsque le graphe est suffisamment dense.
- Elle permet d'implémenter assez facilement certains algorithmes.
- Deux arcs ayant les mêmes extrémités ne peuvent pas être représentés avec cette matrice



Propriétés

Dans un graphe orienté, on distingue :

Demi degré extérieur = nombre d'arcs dont s est l'origine
 = degré sortant
 $= d^+(s) = |\Gamma(s)|$
 $= \sum_{t=1}^{t=n} m_{st}$

Demi degré intérieur = nombre d'arcs dont s est l'extrémité
 = degré entrant
 $= d^-(s) = |\Gamma^{-1}(s)|$
 $= \sum_{t=1}^{t=n} m_{ts}$

$$p = |A| = \sum_{s=1}^{s=n} d^+(s) = \sum_{s=1}^{s=n} d^-(s)$$

Théorème (voir TD)

Soit $G=(S, A)$ un graphe orienté de matrice d'adjacence A .

Pour tout entier $k \in \mathbb{N}^*$ on a : $A^k = (m_{st}^k)$

(m_{st}^k) = nombre de chemins de longueur k du sommet s au sommet t

Démonstration par récurrence sur k

Représentations des graphes

Pour décrire un graphe G un certain nombre de représentations peuvent être utilisées. En pratique, elles ne sont généralement pas équivalentes du point de vue de l'efficacité des algorithmes.

On distingue deux grandes familles de représentation :

- Matrice d'adjacence ou ses dérivées :

On privilégie le fait qu'un graphe est un ensemble de sommets. Cette représentation correspond au cas où l'ensemble des sommets du graphe n'évolue pas.

- Matrice d'incidence ou ses dérivées :

On privilégie le fait qu'un graphe est un ensemble d'arcs (resp. arêtes).

Représentation à partir de la matrice d'adjacence

La matrice d'adjacence permet de décrire soit des 1-graphes (orientés) soit des graphes simples (non orientés).

Remarque

La quantité d'informations nécessaire est égale à n^2 ($n=|S|$)

Avantages

- La représentation matricielle est pratique pour tester l'existence d'un arc (resp. arête) entre deux sommets en $O(1)$.
- Il est facile, d'ajouter ou de retirer un arc (resp. arête)
- Il est également facile de parcourir tous les successeurs ou prédécesseurs d'un sommet s donné.

```
Pour t variant de 1 à n faire
    Si g.mat[s, t] alors
        traiter(t)
    Finsi
Finpour
```

On effectue $n=|S|$ tests pour trouver les successeurs d'un sommet s , même s'il n'y a pas de successeurs.

Inconvénients

- Consultation de la matrice nécessite un temps de l'ordre de n^2
- Cette représentation exige un espace mémoire de l'ordre de n^2 : quel que soit le nombre d'arcs (resp. arêtes), la place mémoire nécessaire est de l'ordre de $O(n^2)$.
- Dans le cas des graphes peu denses

$\text{Card}[A] \ll (\text{Card}(S))^2$ pour les graphes orientés

$\text{Card}[A] \ll 1/2(\text{Card}(S)(\text{Card}(S)+1))$ pour les graphes orientés.

→ il y a donc une perte importante d'information

Solution → Décrire uniquement les termes non nuls de la matrice :
Matrice Creuse, Liste D'adjacence, File des successeurs, liste principale, etc

File des successeurs d'un graphe

Rappel : Fonction successeur

$$\Gamma : S \rightarrow \mathfrak{S}(S)$$

$$s \rightarrow \Gamma(s) = \{ t \in S / (s, t) \in A \}$$

= Ensemble des successeurs de s

Soit $G=(S, A)$ un graphe tel que $S=\{1, 2, \dots, n\}$

$\Gamma(s)$ =est la liste des successeurs du sommet $s \in S(G)$.

FS = File des successeurs

=est un tableau linéaire contenant successivement les listes **non**

vides des successeurs $\Gamma(1), \Gamma(2), \dots, \Gamma(n)$.

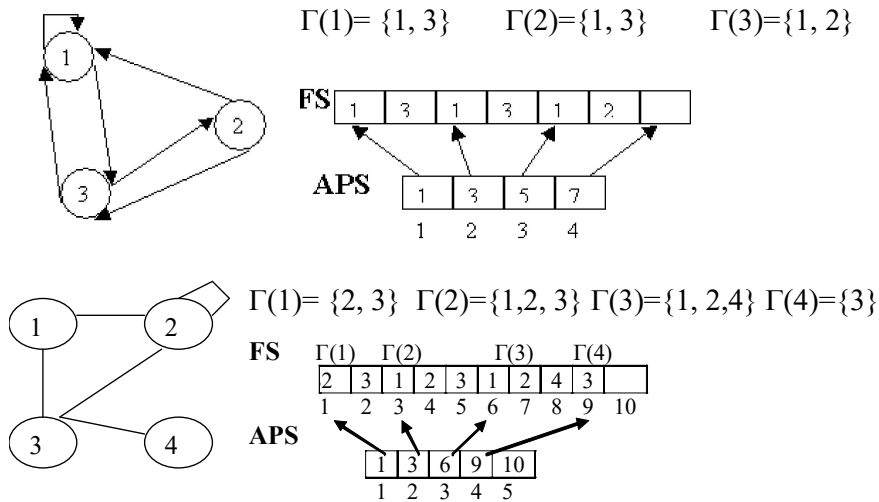
APS = Adresse (=indice) du premier successeur

= Tableau linéaire indicé par les numéros des sommets.

$\text{APS}[s]$ = contient l'adresse (=indice) du premier successeur du sommet s.

Pour faciliter la gestion des successeurs (simplifier les calculs) ; on insère un **sommet virtuel** $s=n+1$ tel que $\text{APS}[n+1]=p+1$.

Avantage : Accès directe à la liste d'adjacence d'un sommet s.

Exemple**Remarque :**

Les flèches ne font pas partir de la structure.

Déclaration de la structure

```
typedef struct{
    int nbSom ;
    int nbArcs ;
    int *FS ;
    int *APS ;
    //TCout *Cout ;
} FileSucc ;
```

Remarques

- ❖ L'ensemble des informations relatives au sommet s est contenu entre les cases $APS[s]$ et $APS[s+1]$.
 $d^+[s] = |\Gamma(s)| = APS[s+1] - APS[s]$ cas orienté
 $d[s] = |\Gamma(s)| = APS[s+1] - APS[s]$ cas non orienté
- ❖ Le tableau APS est de taille $n+1=|S|+\text{Sommet virtuel}$.
- ❖ Le tableau FS est de taille $p=|A|$ si le graphe est orienté et de taille $\leq 2p$ dans le cas non orienté.
- ❖ Lorsque le graphe est valué, c'est à dire qu'à chaque arc (resp. arête) $a \in A$ on associe un poids ou une valeur $v(a)$, la valeur peut être stockée dans un tableau **COUT**, en correspondance biunivoque avec FS.
- ❖ Cette représentation est **avantageuse** pour les graphes contenant peu d'arc. Par exemple les graphes planaires.
- ❖ L'inconvénient de cette représentation est qu'elle est statique.
 Par exemple, le partage d'un graphe en plusieurs sous graphes.

❖ Solution : représentation par les pointeurs.

Représentation par liste d'adjacences

Soit $G(S, A)$ un graphe.

Cette représentation consiste à représenter l'ensemble des sommets du graphe par un tableau de listes chaînées de taille $n=|S|$.

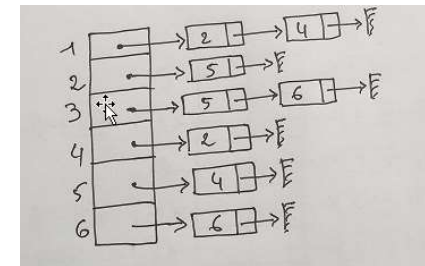
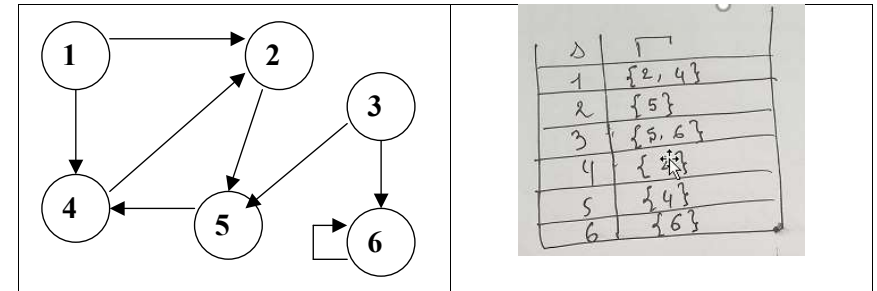
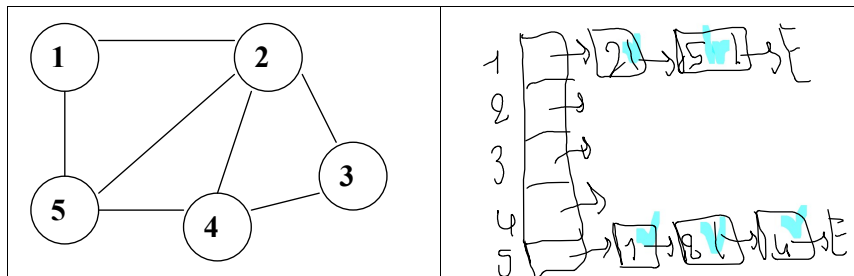
Ce tableau est indicé par les numéros de sommets. Et associé à chaque sommet la liste de ses successeurs rangés dans un certain ordre.

Ces listes sont appelées **listes d'adjacences**.

La liste chaînée est une structure composée de 3 membres :

- 1) Numéro du sommet successeur
- 2) Valeur de l'arc dans le cas d'un graphe valué
- 3) Adresse du sommet suivant

Exemple



Déclaration de la structure : représentation chaînée

<pre>typedef struct{ int nbSom ; Liste *tabAdj ; } ListAdjacence ;</pre>	<pre>typedef struct cellule{ int somSucc ; TCout cout ; struct cellule *suiv ; } *Liste ;</pre>
--	---

Occupation mémoire

Soit $G(S, A)$ un graphe avec $n=|S|$ et $p=|A|$

Occupation mémoire est de l'ordre $\theta(n+p)$ si G est orienté et $\theta(n+2p)$ si G est non orienté.

Traitement d'un successeur d'un sommet s :

```

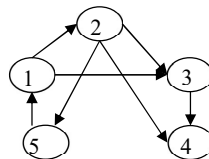
l ← tabAdj[s]
Tantque non estVide(l) faire
    traiter (somSucc(l))
    l ← suivant(l)
Fintq
    
```

- L'accès aux successeurs d'un sommet s est de l'ordre $O(d^+(s))$
- L'accès aux prédécesseurs d'un sommet s nécessite le parcours de toutes les listes d'adjacences $O(n+p)$ à comparer avec n dans le cas d'une représentation par matrice d'adjacence.

Une solution dans le cas où l'on a besoin de connaître les prédécesseurs d'un sommet est de maintenir, en plus de la liste d'adjacence des successeurs, la liste d'adjacence des prédécesseurs.

Représentation Contigüe de la structure

numSom	Degré	Liste d'adjacence
1	2	2 3
2	3	3 4 5
3	1	4
4	0	
5	1	1



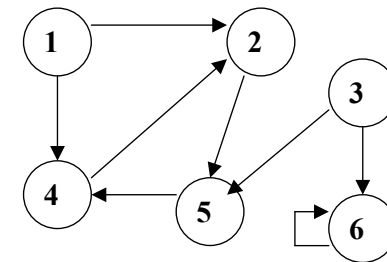
La colonne des degrés représente uniquement les degrés-sortants et la colonne « Liste d'adjacence » sont les successeurs de s_i (ensemble d'adjacence du sommet s_i) avec $1 \leq i \leq n$.

Listes adjacences Successeurs & Prédécesseurs

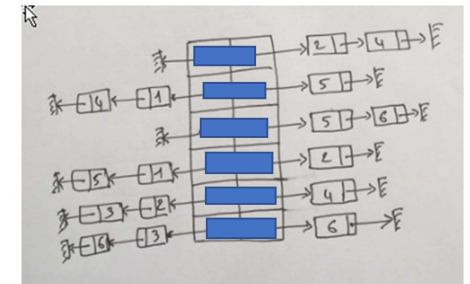
Un graphe peut être représenté par un tableau de listes.

Pour chaque sommet, en associé une liste de successeurs et une liste de prédécesseurs.

Exemple



s	$\Gamma^+(s)$	$\Gamma^-(s)$
1	{2, 4}	{}
2	{5}	{1, 4}
3	{5, 6}	{}
4	{2}	{1, 5}
5	{4}	{2, 3}
6	{6}	{3, 5}



Déclaration de la structure : représentation chaînée

Version 1

```
typedef struct celluleSucc{
    int somSucc ;
    struct celluleSucc *suivSucc;
} *ListeSucc;
```

```
typedef struct cellulePred{
    int somPred ;
    struct celluleSucc *suivPred;
} *ListePred;
```

```
typedef struct {
    ListeSucc lSucc;      // Liste des successeurs
    ListePred lPred;      // Liste des Prédécesseurs
} ListeSuccPred;
```

```
typedef struct{
    int nbSom ;
    ListeSuccPred *tabAdjSP;
} ListAdjSuccPred;
```

Version 2

```
typedef struct celluleAdj{
    int somAdj ;
    struct celluleAdj *suivAdj;
} *Liste;
```

```
typedef struct {
    Liste lSucc;          // Liste des successeurs
    Liste lPred;          // Liste des Prédécesseurs
} ListeSuccPred;
```

```
typedef struct{
    int nbSom ;
    ListeSuccPred *tabAdjSP;
} ListAdjSuccPred;
```

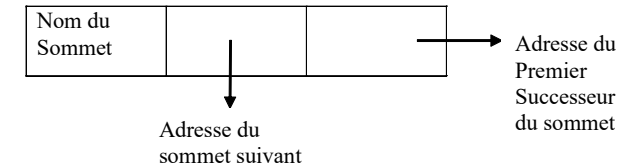
Remarques

- Elle est très bien adaptée au parcours du graphe, aussi bien en sens direct qu'en sens indirect.
 - Cette représentation est souple : l'ajout et la suppression de sommets ou d'arcs sont plus aisés qu'avec les représentations matricielles.
 - Représentation est nettement plus efficace au niveau de la mémoire occupée que les représentations matricielles.
- De nombreuses variantes et adaptations de ces implémentations sont possibles et à choisir en fonction du sujet à traiter :
- Une possibilité intéressante est de rendre circulaire les listes linéaires associées aux listes successeurs (resp. listes prédécesseurs).
- On peut également rendre les listes bidirectionnelles.

Représentation par pointeurs= Liste principale

Les sommets du graphe sont associés en une liste linéaire appelée **liste principale**.

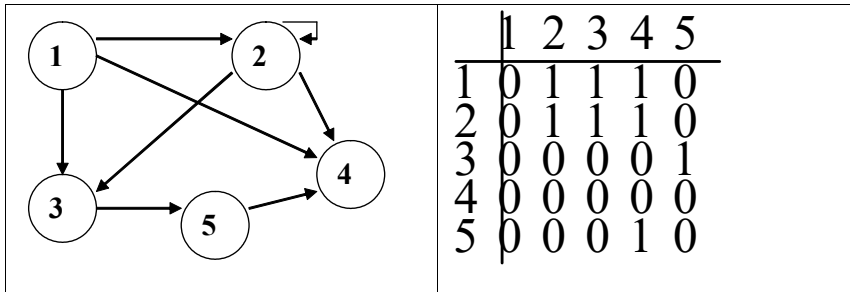
Chaque sommet du graphe étant représenté dans cette liste par une structure comportant 3 membres :



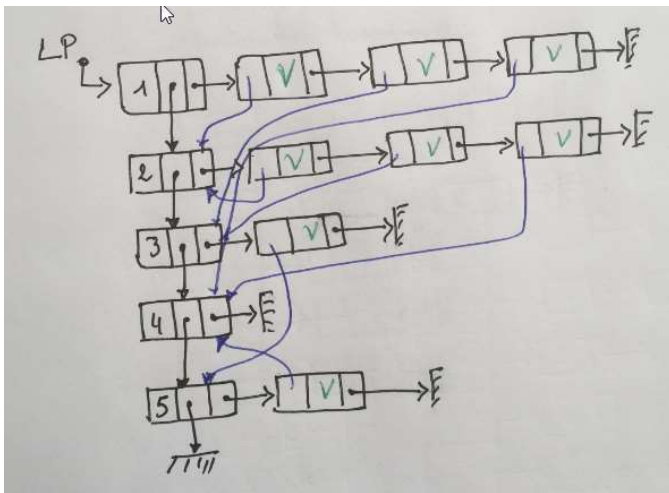
- Nom du sommet
- Un pointeur sur le sommet suivant
- Un pointeur sur la liste des successeurs de ce sommet.

Chaque successeur t de s est représenté par une structure comportant 3 membres :

- Un pointeur vers la structure t dans la liste principale
- La valeur de l'arc (s, t) dans le cas d'un graphe valué.
- Pointeur vers le sommet suivant de cette liste de successeurs.

Exemple

Δ	$\Gamma(\Delta)$
1	{2, 3, 4}
2	{2, 3, 4}
3	{5}
4	{ }
5	{4}

Déclaration de la structure

```

typedef struct cellule *ListeSucc ;
typedef struct listPrincipale *ListePrincipale ;

struct listPrincipale {

    int numSom ;

    ListePrincipale suivLP ;

    ListeSucc lSucc ;

} ;

struct cellule{

    ListePrincipale succ ;

    TCout cout ;

    ListeSucc suivLS ;

}

```

Représentation de la matrice d'adjacence par une matrice creuse

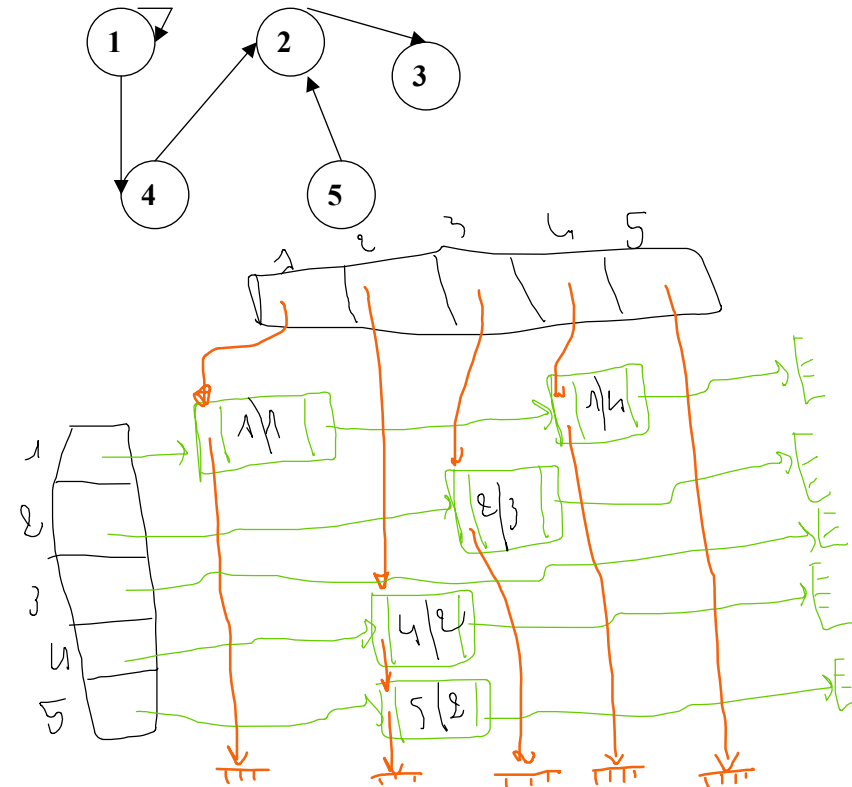
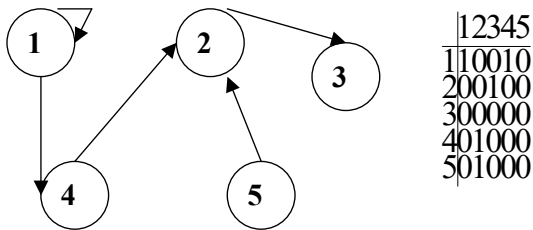
Définition

Une matrice est qualifiée de creuse s'elle comporte majoritairement des coefficients nuls ou si le pourcentage des coefficients non nul est faible.

Principe

On ne représente que les termes non nuls de la matrice. Pour cela on utilise une représentation dynamique par pointeur.

Exemple



Structure en langage C

Version 1 : avec les notations du matrice en math

```
typedef struct cellule {  
    int numlig;  
    int numcol;  
    TCout cout;  
    struct cellule *ligSuiv;  
    struct cellule *colSuiv ;  
}* Liste ;
```

```
typedef struct matrice{  
    int nblig ;  
    int nbcol;  
    Liste *ligne;  
    Liste *colonne;  
} MatriceCreuse;
```

Version 2 : avec les notations d'un graphe

```
typedef struct cellule {  
    int somSucc;  
    int somPred;  
    TCout cout;  
    struct cellule *succSuiv;  
    struct cellule *predSuiv ;  
}* Liste ;
```

```
typedef struct matrice{  
    int nbSom ;  
    Liste *tabSucc;  
    Liste *tabPred;  
} MatriceCreuse;
```

Représentation statique

Première solution :

Une matrice creuse peut être représentée avec un tableau de coefficients et deux tableaux d'indices (des entiers entre 1 et $n=|S|$).

Exemple

$$A = \begin{bmatrix} 3 & 0 & 1 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 \\ 0 & 7 & 5 & 5 & 0 \\ 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 6 & 5 \end{bmatrix}$$

Ligne =

1	1	2	3	3	3	4	5	5
1	3	2	2	3	4	5	4	5

 Colonne =

1	3	2	2	3	4	5	4	5
---	---	---	---	---	---	---	---	---

 Coef =

3	1	4	7	5	5	2	6	5
---	---	---	---	---	---	---	---	---

Deuxième solution :

On peut représenter une matrice creuse par une liste chaînée, contenant uniquement les coefficients non nuls avec leurs indices.

Les coefficients sont rangés par indice croissant (ligne puis colonne).

Pour connaître la dimension de la matrice, le coefficient d'indice maximal est précisé dans la liste chaînée ; même si celui-ci est nul.

Exemple

$$A = \begin{bmatrix} 0 & 2 & 0 & 0 & 0 \\ 1 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 6 & 0 \end{bmatrix}$$

L →

1,2
2

 →

2,1
1

 →

2,3
3

 →

4,4
6

 →

4,5
0

 → 

Représentation à partir de la matrice d'incidence

Rappel: Définition d'un Multigraphe

Un multigraphe est un graphe pour lequel il peut exister plusieurs arêtes entre deux sommets s et t données.



La matrice d'incidence sommets-arcs ou sommets-arêtes permet de décrire parfaitement la structure d'un multigraphe à condition qu'il soit sans boucles.

Remarque : il n'y a que deux termes non nuls par colonnes → Représenter l'information sous forme plus condensée.

a) Représentation colonne par colonne

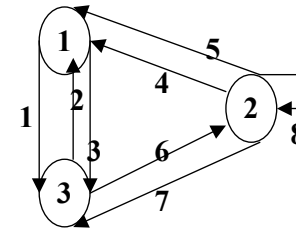
On utilise deux tableaux $\alpha[]$ et $\beta[]$ de dimension $p=|A|$ donnant pour chaque arc ou arête $a=1..p$ les numéros $\alpha[a]$ et $\beta[a]$ des extrémités.

si G est orienté alors

$\alpha[a]$ est l'origine de a

$\beta[a]$ est l'extrémité de a

Exemple

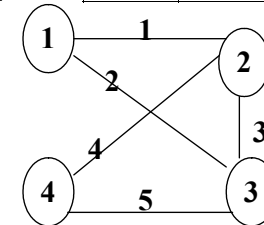


$$\alpha =$$

1	3	1	2	2	3	2	2
---	---	---	---	---	---	---	---

$$\beta =$$

3	1	3	1	1	2	3	2
---	---	---	---	---	---	---	---



$$\alpha =$$

1	1	2	2	3
---	---	---	---	---

$$\beta =$$

2	3	3	4	1
---	---	---	---	---

Remarques

- ❖ Lorsque le graphe est valué, il suffit d'ajouter un tableau **COUT[]** de dimension p en correspondance biunivoque avec les tableaux $\alpha[]$ et $\beta[]$.
- ❖ Contrairement à la matrice d'incidence, cette représentation permet de décrire également des multigraphes comportant des boucles.
- ❖ Description de la matrice d'incidence colonne par colonne.

Déclaration de la structure

```
typedef struct{
    // TGraphe indicateur;
    int nbSom ;
    int nbArcs ; // int nbAretes
    int *alpha ;
    int *beta ;
    TCout *Cout ;
} MatIncidColCol ;
```

b) Description ligne par ligne de la matrice d'incidence.

On peut aussi décrire la matrice d'incidence sommets-arcs (resp. Sommets-arêtes) ligne par ligne, en indiquant pour chaque sommet $s \in S(G)$:

La liste $w^+(s)$ des arcs ayant s comme origine (**cas orienté**)

$$w^+(s) = \{a / a=(s, t) \in A(G)\}$$

La liste $w(s)$ des arêtes incidentes à s (**cas non orienté**).

$$w(s) = \{a / a=\{s, t\} \in A(G)\}$$

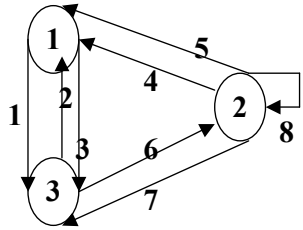
Même principe que la file des successeurs. Mais cette fois, on utilise :

APS=indiqué par sommet donne l'adresse (indice) du premier arc dont le sommet est l'origine (+sommet virtuel),

FS=tableau contenant la liste successive des arcs,

AS=liste des extrémités de chaque arc.

Exemples



$$w^+(1) = \{1, 3\}$$

$$w^+(2) = \{4, 5, 7, 8\}$$

$$w^+(3) = \{2, 6\}$$

APS=

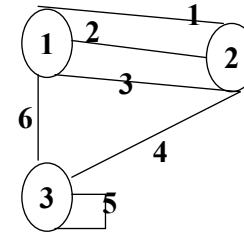
1	3	7	9
---	---	---	---

FS=

1	3	4	5	7	8	2	6
---	---	---	---	---	---	---	---

AS=

3	3	1	1	3	2	1	2
---	---	---	---	---	---	---	---



$$w(1) = \{1, 2, 3, 6\} \quad w(2) = \{1, 2, 3, 4\} \quad w(3) = \{4, 5, 6\}$$

APS=

1	5	9	12
---	---	---	----

FS

1	2	3	6	1	2	3	4	4	5	6
---	---	---	---	---	---	---	---	---	---	---

AS

2	2	2	3	1	1	1	3	2	3	1
---	---	---	---	---	---	---	---	---	---	---

Remarque

Si le graphe est valué, il suffira de disposer un tableau supplémentaire **cout[]** en correspondance biunivoque avec FS et AS.

Déclaration de la structure

```
typedef struct {
    int nbSom ;
    int nbArcs ;
    int *FS ;
    int *APS ;
    int *AS ;
    TCout *Cout ;
} MatIncidLigLig ;
```

Liste d'incidences

Soit $G=(S, A)$ un graphe où $n=|S|$ et $p=|A|$

Cette représentation consiste à représenter l'ensemble des sommets du graphe par un tableau de listes chaînées de taille n .

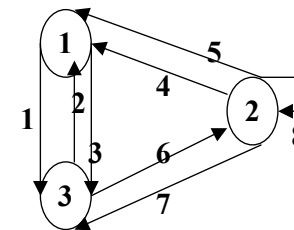
Ce tableau est indicé par les numéros de sommets. Et associé à chaque sommet $s \in S(G)$:

La liste $w^+(s)$ des arcs ayant s comme origine (cas orienté)

$$w^+(s) = \{a / a=(s, t) \in A(G)\}$$

La liste $w(s)$ des arêtes incidentes à s (cas non orienté).

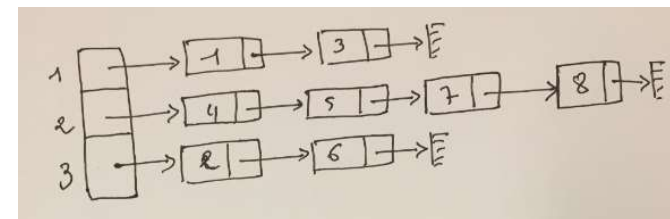
$$w(s) = \{a / a=\{s, t\} \in A(G)\}$$

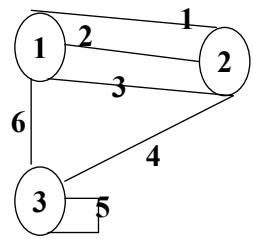
Exemples

$$w^+(1)=\{1, 3\}$$

$$w^+(2)=\{4, 5, 7, 8\}$$

$$w^+(3)=\{2, 6\}$$





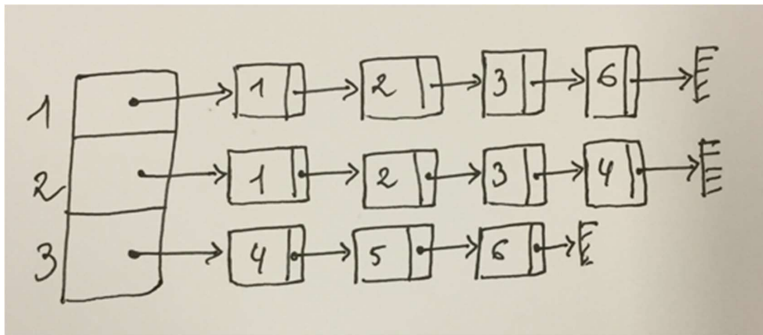
$$w(1) = \{1, 2, 3, 6\}$$

$$w(2) = \{1, 2, 3, 4\}$$

$$w(3) = \{4, 5, 6\}$$

File d'incidences

ETC



Choix entre les différents types de représentations

Le choix se fait en fonction de :

- Densité du graphe

P voisin de n^2 $\rightarrow$ graphe est dense

$P \ll n^2$ $\rightarrow$ peu denses

- Type d'algorithmes que l'on désire utiliser sur le graphe.